

PART 1: Minimum Spanning Tree (Prim's and Kruskal's Algorithm)

Aim

To write a program that finds the Minimum Spanning Tree (MST) of a weighted graph using Prim's and Kruskal's algorithms.

Theory

A Minimum Spanning Tree of a connected, weighted, undirected graph is a subset of $V-1$ edges that connects all vertices with the minimum possible total weight and no cycles.

Prim's algorithm (greedy) grows the tree from a starting vertex, repeatedly adding the minimum-weight edge that connects a visited vertex to an unvisited one; with a min-heap its time complexity is $O(E \log V)$.

Kruskal's algorithm (greedy) sorts all edges by weight and adds each edge if it does not form a cycle, checked using Union-Find (Disjoint Set Union); its time complexity is $O(E \log E)$.

Code

```
class Solution {
public:
    int spanningTree(int V, vector<vector<int>>& edges) {
        unordered_map<int, vector<vector<int>>>m;
        for(int i=0; i<edges.size(); i++){
            int fv=edges[i][0];
            int sv=edges[i][1];
            int w=edges[i][2];
            m[fv].push_back({sv, w});
            m[sv].push_back({fv, w});
        }
        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
        vector<int> vist(V, 0);
        pq.push({0, 0});
        int sum=0;
        while(!pq.empty()){
            auto it=pq.top();
            pq.pop();
            int we=it.first;
            int node=it.second;
            if(vist[node]==1) continue;
            vist[node]=1;
            sum+=we;
            for(auto it:m[node]){
                int adjnode=it[0];
                int adjw=it[1];
                if(vist[adjnode]==0){
                    pq.push({adjw, adjnode});
                }
            }
        }
        return sum;
    }
};
```

Output:

V =

3

E =

3

Edges[[u, v, w]] =

```
0 1 5
1 2 3
0 2 1
```

Your Output:

4

Expected Output:

4

Code:

```

class Solution {
public:
    vector<int>parent;
    vector<int>size;
    void disjointset(int n){
        parent.resize(n);
        size.resize(n);
        for(int i=0;i<n;i++){
            parent[i]=i;
            size[i]=1;
        }
    }
    int findUpPar(int node){
        if(node==parent[node]){
            return node;
        }
        return parent[node]=findUpPar(parent[node]);
    }
    void unions(int u,int v){
        int uparent=findUpPar(u);
        int vparent=findUpPar(v);

        if(uparent==vparent){
            return ;
        }
        else if(size[uparent]<size[vparent]){
            parent[uparent]=vparent;
            size[vparent]+=size[uparent];
        }
        else{
            parent[vparent]=uparent;
            size[uparent]+=size[vparent];
        }
        return;
    }
    int kruskalsMST(int V, vector<vector<int>> &edges) {
        disjointset(V);
        sort(edges.begin(),edges.end(),[](vector<int>&a,vector<int>&b){
            return a[2]<b[2];
        });
        int mcost=0;
        for(int i=0;i<edges.size();i++){
            int p=edges[i][0];
            int q=edges[i][1];
            int cost=edges[i][2];
            if(findUpPar(p)!=findUpPar(q)){
                mcost+=cost;
                unions(p,q);
            }
        }
        return mcost;
    }
};

```

Output

V

3

E

3

edges[]

0 1 5

1 2 3

0 2 1

Your Output:

4

Expected Output:

4

PART 2: Dijkstra's Algorithm (Single Source Shortest Path)

Aim

To write a program that finds the shortest distance and the path from a given source vertex to every other vertex of a weighted graph with non-negative edge weights using Dijkstra's algorithm (graph taken as input from the user).

Theory

Dijkstra's algorithm is a greedy method that finds the shortest path from a single source to all other vertices in a graph with non-negative edge weights.

It maintains a distance array initialised to infinity (0 for the source) and a priority queue; the vertex with the smallest tentative distance is repeatedly extracted and its neighbours are relaxed. A parent array is used to reconstruct the path. Time complexity is $O((V+E) \log V)$ with a min-heap.

Code

```
class Solution {
public:
    vector<int> dijkstra(int V, vector<vector<int>> &edges, int src) {
        unordered_map<int, vector<vector<int>>>m;
        for(int i=0; i<edges.size(); i++){
            int fv=edges[i][0];
            int sv=edges[i][1];
            int we=edges[i][2];

            m[fv].push_back({sv, we});
            m[sv].push_back({fv, we});
        }
        vector<int>dist(V, INT_MAX);
        dist[src]=0;
        vector<int>vist(V, 0);
        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>>pq;
        pq.push({0, src});
        while(!pq.empty()){
            auto it=pq.top();
            pq.pop();
            int node=it.second;
            int we=it.first;
            if(vist[node]==1)continue;
            vist[node]=1;
            for(auto it:m[node]){
                int adjnode=it[0];
                int adjw=it[1];
                if(vist[adjnode]==0){
                    if(dist[adjnode]>dist[node]+adjw){
                        dist[adjnode]=dist[node]+adjw;
                        pq.push({dist[node]+adjw, adjnode});
                    }
                }
            }
        }
        return dist;
    }
};
```

Output

V =

3

E =

3

edges[][] =

```
0 1 1
1 2 3
0 2 6
```

src =

2

Your Output:

4 3 0

Expected Output:

4 3 0